

Google Career — Day 4

Interview Questions & Detailed Answers — Textbook Edition

Scope. This textbook consolidates the Day 4 interview bank into detailed Q&A.; It covers the layered FastAPI document service, Pydantic contracts, service/repository separation, in-memory persistence, API testing, Sliding Window DSA, engineering decisions, Git discipline, and project/system-design follow-ups. It does not claim PostgreSQL, Redis, Docker, Kubernetes, GCP, RAG, agents, React, authentication, or PDF processing are complete.

How to Use This Book

Answer each question aloud before reading the model answer. For DSA, always state Pattern → Invariant → Brute Force → Optimized Approach → Complexity → Edge Cases. For engineering questions, state WHAT → WHY → HOW → TRADE-OFF → FAILURE → TEST → SCALE → FOLLOW-UP.

Table of Contents

Part I — Project Architecture — 9 detailed questions
Part II — Document API — 10 detailed questions
Part III — FastAPI — 10 detailed questions
Part IV — Pydantic — 10 detailed questions
Part V — Service Layer — 10 detailed questions
Part VI — Repository — 10 detailed questions
Part VII — In-Memory Storage — 10 detailed questions
Part VIII — Error Handling — 10 detailed questions
Part IX — API Testing — 10 detailed questions
Part X — End-to-End API — 10 detailed questions
Part XI — API Design and Production Follow-ups — 10 detailed questions
Part XII — Sliding Window Theory — 10 detailed questions
Part XIII — Fixed Window: Maximum Average Subarray I — 9 detailed questions
Part XIV — Variable Window — 10 detailed questions
Part XV — Sliding Window vs Two Pointers — 10 detailed questions
Part XVI — Longest Substring Without Repeating Characters — 15 detailed questions
Part XVII — Maximum Average Subarray I: Follow-ups — 7 detailed questions
Part XVIII — Minimum Size Subarray Sum — 10 detailed questions
Part XIX — Complexity — 10 detailed questions
Part XX — Edge Cases — 10 detailed questions
Part XXI — Code Explanation — 10 detailed questions
Part XXII — Engineering Decisions — 10 detailed questions
Part XXIII — Project-Based Google Follow-ups — 10 detailed questions
Part XXIV — System Design Follow-ups — 10 detailed questions
Part XXV — Git and Engineering Practice — 10 detailed questions
Part XXVI — Scenarios — 10 detailed questions
Part XXVII — Mock Interview — 10 detailed questions
Appendix A — Rapid-Fire Revision — 40 prompts
Appendix B — Top 20 Questions to Master
Final Day 4 Mastery Standard

Part I — Project Architecture

Q1. What is the architecture of the Day 4 document service?

Answer. The architecture is layered: HTTP Client → FastAPI/API Layer → DocumentService → DocumentRepository → In-Memory Storage. FastAPI owns HTTP concerns, the service owns application behavior and business rules, and the repository owns persistence operations. This separation is the central engineering lesson of Day 4.

Q2. Why use API → Service → Repository?

Answer. It separates concerns and creates explicit boundaries. Routes stay thin, business logic is reusable and independently testable, and persistence can later move from memory to PostgreSQL without rewriting the application.

Q3. What is separation of concerns?

Answer. It means each component has a focused responsibility. In this project, HTTP handling, business behavior, and persistence are deliberately separated.

Q4. Why should business logic not live in FastAPI routes?

Answer. Routes are transport-specific. If business rules live there, they become coupled to HTTP and are harder to reuse from tests, background jobs, CLI commands, or future agent tools. The service layer provides a reusable application boundary.

Q5. What is the role of DocumentService?

Answer. It exposes operations such as `create_document`, `get_document`, `list_documents`, and `delete_document`. It owns application behavior and business rules while remaining independent of HTTP and storage technology.

Q6. What is the role of DocumentRepository?

Answer. It abstracts persistence operations such as `create`, `get`, `get_all`, and `delete`. It should not know about HTTP status codes or FastAPI request objects.

Q7. Why is in-memory storage acceptable on Day 4?

Answer. The purpose is to validate architecture, API behavior, validation, and tests before introducing database complexity. The repository boundary makes the storage implementation replaceable.

Q8. How would you explain the architecture in a Google interview?

Answer. I would say: 'I separated HTTP, application logic, and persistence. FastAPI validates and translates HTTP, DocumentService owns business behavior, and DocumentRepository owns persistence. The current repository is in memory intentionally; PostgreSQL is the next persistence step.'

Q9. What is the most important architectural lesson?

Answer. A working endpoint is only the beginning. A maintainable backend needs explicit responsibilities, contracts, errors, tests, and a clear path for evolution.

Part II — Document API

Q1. What endpoints should Day 4 support?

Answer. POST /documents, GET /documents, GET /documents/{id}, and DELETE /documents/{id}, plus the basic root and health endpoints.

Q2. What happens during POST /documents?

Answer. FastAPI receives the request, Pydantic validates it, the API calls DocumentService, the service delegates persistence to DocumentRepository, the repository stores the document, and a response model serializes the result.

Q3. What should GET /documents return when empty?

Answer. A successful response containing an empty list. The collection exists even when it has no members; 404 is appropriate for a missing specific resource.

Q4. Why is GET /documents/{id} different?

Answer. It requests one specific resource. If that resource does not exist, the API should communicate that as a not-found condition, normally HTTP 404.

Q5. What should DELETE /documents/{id} do?

Answer. The route delegates to the service, the service delegates to the repository, and the repository removes the resource. The route should not manipulate storage directly.

Q6. What happens when DELETE targets a missing document?

Answer. The application should represent the missing-resource condition and the API layer should translate it to HTTP 404.

Q7. Why should routes not access the dictionary directly?

Answer. That would couple HTTP code to one storage implementation and bypass the repository boundary, making migration and testing harder.

Q8. What is an API contract?

Answer. It defines methods, paths, request schemas, response schemas, status codes, and error behavior. It is the agreement between the client and server.

Q9. Why are response models important?

Answer. They make the output contract explicit, validate serialization, document the API, and reduce accidental exposure of internal fields.

Q10. What is a resource-oriented API?

Answer. It models entities as resources addressed by URLs and uses HTTP methods for operations such as create, read, and delete.

Part III — FastAPI

Q1. What is FastAPI?

Answer. A Python web framework for APIs that uses type hints, validation, automatic documentation, dependency injection, and asynchronous capabilities.

Q2. Why is FastAPI useful here?

Answer. It gives the project a clear HTTP boundary, typed request/response contracts, validation, OpenAPI documentation, and a convenient testing surface.

Q3. What is Uvicorn?

Answer. Uvicorn is an ASGI server that runs applications such as FastAPI. FastAPI defines the application; Uvicorn provides the server runtime.

Q4. What is ASGI?

Answer. ASGI is the interface between Python web servers and asynchronous-capable applications. It enables frameworks such as FastAPI to handle HTTP workloads through an application-server boundary.

Q5. What is routing?

Answer. Routing maps an HTTP method and URL path to an endpoint handler.

Q6. What is dependency injection in FastAPI?

Answer. It lets an endpoint declare dependencies such as services or repositories while FastAPI supplies them. This improves composition and testability.

Q7. Why use type hints?

Answer. They communicate intended types to developers and tooling and allow FastAPI to derive validation and documentation information.

Q8. What is HTTP 404?

Answer. It means the requested resource was not found. Day 4 uses it primarily for missing document IDs.

Q9. 400 vs 404?

Answer. 400 describes an invalid request; 404 describes a missing requested resource. They represent different client-visible conditions.

Q10. Why keep HTTP details in the API layer?

Answer. HTTP is a transport boundary. Keeping it there prevents application logic from depending on one transport and makes services reusable.

Part IV — Pydantic

Q1. What is Pydantic?

Answer. A Python library for parsing and validating structured data using type annotations. FastAPI uses Pydantic models for request and response schemas.

Q2. Why use DocumentCreate?

Answer. It defines the input contract for creating a document and prevents clients from controlling fields that should be generated or managed by the server.

Q3. Why use DocumentResponse?

Answer. It defines the public output contract and separates returned data from creation input and internal storage representation.

Q4. Why separate API schemas from database models?

Answer. API models are external contracts; database models are persistence representations. Separating them reduces coupling and makes database evolution safer.

Q5. What happens with invalid Pydantic input?

Answer. FastAPI rejects it at the API boundary with a validation response before the normal service workflow executes.

Q6. Why validate at the boundary?

Answer. Malformed data is rejected before it contaminates deeper layers, reducing duplicated defensive checks and making behavior predictable.

Q7. Should every business rule live in Pydantic?

Answer. No. Pydantic is appropriate for data shape and boundary validation. State-dependent business rules belong in the service layer.

Q8. Validation vs business logic?

Answer. Validation checks whether data conforms to expected structure and constraints; business logic determines what the application should do with valid data.

Q9. How does Pydantic help documentation?

Answer. FastAPI can use model definitions to generate OpenAPI schemas and interactive documentation.

Q10. What is the interview takeaway?

Answer. Use Pydantic to make boundaries explicit, not merely because it is convenient for parsing JSON.

Part V — Service Layer

Q1. What belongs in DocumentService?

Answer. Application operations and business rules: create, get, list, delete, and decisions about application-level outcomes.

Q2. Why have a service layer in a small project?

Answer. It prevents routes from becoming fat controllers and creates a stable boundary for future background jobs, tools, agents, and other interfaces.

Q3. How should the service handle a missing document?

Answer. It should communicate an application-level not-found condition without depending on HTTP. The API layer translates that condition to 404.

Q4. Should the service know SQL?

Answer. No. SQL and database mechanics belong behind the repository boundary.

Q5. How would you unit test the service?

Answer. Inject a fake or controlled repository, call the service operation, and assert business behavior and repository interactions without requiring HTTP or a real database.

Q6. What is orchestration?

Answer. Coordinating multiple application operations to implement one higher-level workflow while keeping transport-specific details outside the service.

Q7. What is a fat controller?

Answer. A controller or route that contains too much business logic, persistence logic, and orchestration. It becomes hard to test and maintain.

Q8. What is a thin controller?

Answer. A controller that performs HTTP-specific work and delegates application behavior to services.

Q9. How does this help future agents?

Answer. Agent tools can invoke application operations through the service boundary rather than duplicating business rules or manipulating the database directly.

Q10. Strong interview answer?

Answer. The service is the application boundary: independent of HTTP and storage, reusable, and easy to test.

Part VI — Repository

Q1. What is the repository pattern?

Answer. An abstraction around persistence operations that lets application code work with data without depending directly on storage mechanics.

Q2. Why use a repository boundary?

Answer. It gives the service a stable persistence contract and makes storage replacement and testing easier.

Q3. What methods are needed?

Answer. Conceptually create, get, get_all, and delete. Exact signatures can vary; responsibilities should remain clear.

Q4. Should repositories return HTTP responses?

Answer. No. They should return persistence/application results or errors; HTTP translation belongs in the API layer.

Q5. Should repositories know FastAPI Request objects?

Answer. No. That would couple persistence to the web framework.

Q6. How would you replace in-memory storage with PostgreSQL?

Answer. Keep the repository contract stable and implement it using PostgreSQL, migrations, transactions, and connection management. Most service/API logic can remain unchanged.

Q7. Repository vs database?

Answer. A database is a storage system; a repository is an application abstraction over persistence operations.

Q8. What is persistence leakage?

Answer. Database-specific details escaping into service logic or public API contracts.

Q9. How does repository abstraction help testing?

Answer. A fake or in-memory implementation can be injected so service tests remain fast and deterministic.

Q10. What is the trade-off?

Answer. Abstraction adds code, but it can pay off when persistence changes, reuse and testing matter, and the system is expected to grow.

Part VII — In-Memory Storage

Q1. Why use an in-memory dictionary?

Answer. It is simple, fast, deterministic, and avoids database setup while the architecture is being established.

Q2. What are its limitations?

Answer. Data disappears on restart, state is not shared across instances, durability is absent, and memory limits the dataset.

Q3. What happens with two instances?

Answer. Each instance has its own state, so clients can observe inconsistent data depending on which instance handles the request.

Q4. Why not PostgreSQL on Day 4?

Answer. Day 4 focuses on clean boundaries, API behavior, validation, and tests. PostgreSQL is the next planned dependency.

Q5. How does memory help tests?

Answer. It makes test setup and execution fast and deterministic.

Q6. What consistency does one process provide?

Answer. It maintains its own local state, but provides no distributed consistency across replicas.

Q7. Biggest production risk?

Answer. Data loss and lack of shared state.

Q8. How would storage become durable?

Answer. Use PostgreSQL behind the repository boundary, with migrations, transactions, pooling, backups, indexes, and monitoring.

Q9. Where could Redis fit later?

Answer. For justified use cases such as caching, queues, rate limiting, or ephemeral shared state. It should be introduced because a requirement demands it.

Q10. Is Day 4 storage production-ready?

Answer. No. It is intentionally a learning-stage implementation; production needs durable shared persistence.

Part VIII — Error Handling

Q1. Why is error handling part of API design?

Answer. Clients need predictable semantics for validation failures, missing resources, conflicts, authentication failures, and server errors.

Q2. How does a missing document become 404?

Answer. A lower layer represents the not-found condition; the API layer translates it into the HTTP 404 contract.

Q3. Why not return 500?

Answer. Missing resources are expected client-visible outcomes, not unexpected server failures.

Q4. Why avoid returning raw internal exceptions?

Answer. They may leak implementation details or sensitive information and create unstable client contracts.

Q5. What is exception translation?

Answer. Converting an internal/application error into the appropriate external representation, such as mapping `DocumentNotFoundError` to HTTP 404.

Q6. Where should errors be logged?

Answer. At appropriate application boundaries with enough context for diagnosis while avoiding secrets and sensitive information. Deeper observability is part of later production hardening.

Q7. Expected vs unexpected errors?

Answer. Expected errors are known outcomes such as invalid input or missing resources. Unexpected errors indicate defects or unanticipated failures.

Q8. What makes an error contract stable?

Answer. Consistent status codes, structured fields, predictable semantics, and safe messages.

Q9. How should errors be tested?

Answer. Assert the status code and relevant response structure for validation and missing-resource scenarios.

Q10. Interview lesson?

Answer. Failures are part of system behavior and must be designed, translated, tested, and eventually observed.

Part IX — API Testing

Q1. Why test APIs if the code is small?

Answer. Tests protect the external contract and make refactoring safer.

Q2. What should Day 4 API tests cover?

Answer. GET /, GET /health, valid and invalid POST /documents, GET collection, GET existing and missing IDs, and DELETE existing and missing IDs.

Q3. What is an API integration test?

Answer. It exercises the application through an HTTP-like boundary and verifies behavior across multiple layers.

Q4. Unit test vs API test?

Answer. A unit test isolates a component such as the service; an API test validates the external contract and several internal layers together.

Q5. Why test invalid POST?

Answer. Validation is part of the API contract and must be protected against regression.

Q6. Why test an empty collection?

Answer. It verifies that an empty collection is a successful state rather than a not-found error.

Q7. Why test missing GET?

Answer. It verifies the 404 contract and exception translation.

Q8. Why test deletion twice?

Answer. The second deletion verifies the state transition and missing-resource behavior.

Q9. What is regression protection?

Answer. Tests detect unintended changes to previously correct behavior.

Q10. Best testing mindset?

Answer. Test observable behavior and contracts, not implementation details, so tests survive storage replacement.

Part X — End-to-End API

Q1. Walk through GET /documents/{id}.

Answer. HTTP request → route matching/path validation → DocumentService.get_document → repository get → storage lookup → result → response model → HTTP response.

Q2. Walk through missing GET.

Answer. Repository cannot find the resource → application-level not-found condition → API translation → HTTP 404.

Q3. Walk through DELETE.

Answer. HTTP DELETE → API route → service delete → repository delete → storage mutation → successful response.

Q4. What happens before business logic on invalid input?

Answer. FastAPI/Pydantic validation rejects the request at the boundary.

Q5. How does a response model protect the contract?

Answer. It constrains and documents the returned shape.

Q6. How debug a failing API test?

Answer. Start with method/path/input/expected output, then trace API → service → repository → storage and find the first divergence.

Q7. How debug unexpected 404?

Answer. Check creation state, identifier, repository retrieval, dependency/store instance, and exception translation.

Q8. How debug DELETE followed by GET still finding data?

Answer. Trace deletion, verify the same identifier and repository instance are used, and check for duplicated storage state.

Q9. Why explain end-to-end flow in interviews?

Answer. It demonstrates understanding of the system rather than memorization of framework syntax.

Q10. What is the future role?

Answer. It establishes the backend foundation for ingestion, retrieval, impact analysis, test generation, and reporting.

Part XI — API Design and Production Follow-ups

Q1. What makes an API maintainable?

Answer. Explicit contracts, consistent semantics, validation, errors, tests, security, observability, and an evolution strategy.

Q2. When would you version an API?

Answer. When breaking changes must coexist with existing clients and compatibility cannot otherwise be maintained.

Q3. Why are retries and idempotency important?

Answer. Networks fail and clients retry. Predictable repeated operations reduce duplicate side effects and improve reliability.

Q4. What is idempotency?

Answer. Repeating an operation produces the same intended final state as performing it once.

Q5. What is pagination?

Answer. A way to bound the amount of collection data returned in one request.

Q6. Why paginate documents at scale?

Answer. Large responses increase latency, memory, network usage, and backend load.

Q7. What is filtering?

Answer. Allowing clients to request only resources matching defined criteria.

Q8. Authentication vs authorization?

Answer. Authentication establishes who the caller is; authorization determines what that caller may do.

Q9. Why consider security early?

Answer. Security affects contracts, validation, logging, permissions, and architecture, not just a final middleware layer.

Q10. What is not production-ready yet?

Answer. Durable persistence, auth, containerization, CI/CD, full observability, security hardening, cloud deployment, and scalability are future work, not Day 4 completion claims.

Part XII — Sliding Window Theory

Q1. What is Sliding Window?

Answer. A technique for processing a contiguous range while maintaining state incrementally as the range moves.

Q2. When should you suspect it?

Answer. When a problem involves contiguous subarrays/substrings and asks for longest, shortest, maximum, minimum, sum, average, or a window property.

Q3. Fixed vs variable window?

Answer. A fixed window has exactly k elements. A variable window changes length according to a validity condition.

Q4. Why is it often $O(n)$?

Answer. Each element enters and leaves the window a bounded number of times, avoiding repeated rescanning.

Q5. What is an invariant?

Answer. A condition that remains true for the maintained window, such as uniqueness of characters.

Q6. What is brute force?

Answer. Enumerate candidate ranges and repeatedly calculate the property, often producing $O(n^2)$ work.

Q7. What is the optimized approach?

Answer. Maintain only the state needed for the current window and update it when boundaries move.

Q8. Why does contiguity matter?

Answer. The technique depends on the range being a continuous interval.

Q9. Can it work on strings?

Answer. Yes; strings are naturally processed as contiguous character windows.

Q10. Most important interview habit?

Answer. State the window, invariant, expansion rule, contraction rule, answer update, and complexity before coding.

Part XIII — Fixed Window: Maximum Average Subarray I

Q1. Why is this fixed-window?

Answer. Every candidate contains exactly k consecutive elements.

Q2. Why maximize sum rather than average?

Answer. Every candidate is divided by the same positive k , so maximizing the sum also maximizes the average.

Q3. Brute-force complexity?

Answer. Typically $O(nk)$, because each k -element window can be summed from scratch.

Q4. Optimized transition?

Answer. $\text{new_sum} = \text{old_sum} - \text{outgoing} + \text{incoming}$.

Q5. Time complexity?

Answer. $O(n)$.

Q6. Space complexity?

Answer. $O(1)$ additional space.

Q7. Important edge cases?

Answer. $k=1$, $k=n$, negative values, repeated values, and correct initialization.

Q8. Common bug?

Answer. Incorrect outgoing/incoming indices or initializing best sum to zero when all values can be negative.

Q9. How explain without code?

Answer. Compute the first k -sum, slide one position at a time with one subtraction and one addition, track the maximum, then divide by k .

Part XIV — Variable Window

Q1. What is a variable window?

Answer. A contiguous interval whose length changes to preserve or exploit a condition.

Q2. General template?

Answer. Expand right; when invalid, move left until valid; update the objective at the appropriate point.

Q3. When contract?

Answer. When the invariant is violated, or when a valid window can be shortened for a minimization objective.

Q4. Why can it be linear?

Answer. Both pointers move monotonically forward, so total pointer movement is $O(n)$ when state updates are $O(1)$ or amortized $O(1)$.

Q5. Common mistake?

Answer. Updating the answer before restoring validity or shrinking too far.

Q6. How prove correctness?

Answer. State the invariant, show expansion considers all relevant right boundaries, and show contraction restores validity without missing required candidates.

Q7. Why use a frequency map?

Answer. To maintain counts of values currently inside the window.

Q8. Why use a set?

Answer. When only membership is needed, such as tracking unique characters.

Q9. Longest vs shortest window?

Answer. Longest: maximize valid length and shrink only when invalid. Shortest: once valid, shrink repeatedly to minimize length.

Q10. Key mental model?

Answer. The window is maintained state, not just two indices; its invariant drives pointer movement.

Part XV — Sliding Window vs Two Pointers

Q1. Are they the same?

Answer. No. Two Pointers is broader; Sliding Window is a contiguous-range technique that maintains window state.

Q2. Example of Two Pointers?

Answer. Two Sum II uses left/right pointers over a sorted array and moves them based on the current sum.

Q3. Example of Sliding Window?

Answer. Longest Substring Without Repeating Characters maintains a contiguous valid substring.

Q4. Can Sliding Window use two pointers?

Answer. Yes; its left and right boundaries are commonly two pointers.

Q5. How choose?

Answer. Use Sliding Window when a contiguous interval and maintainable invariant define the problem. Use broader Two Pointers when indices coordinate other structures or exploit ordering without a maintained window.

Q6. Common interview trap?

Answer. Calling every two-index algorithm Sliding Window.

Q7. What do both optimize?

Answer. They reduce repeated work through monotonic pointer movement and reuse of information.

Q8. Can Two Pointers be $O(n)$?

Answer. Yes.

Q9. Can Sliding Window be worse than $O(n)$?

Answer. Yes, if the state update or inner work is expensive or pointers are not truly monotonic.

Q10. Interview-quality distinction?

Answer. Two Pointers is the family; Sliding Window is a specialized contiguous-window pattern.

Part XVI — Longest Substring Without Repeating Characters

Q1. What is the problem?

Answer. Find the length of the longest contiguous substring with no repeated characters.

Q2. Why Sliding Window?

Answer. The target is contiguous and the window can maintain a uniqueness invariant.

Q3. Brute force?

Answer. Enumerate substrings and check uniqueness, leading to quadratic or worse work depending on checking.

Q4. Optimized state?

Answer. A set or mapping from character to last-seen index.

Q5. Invariant?

Answer. The active window contains no duplicate characters.

Q6. When does right move?

Answer. It advances to include the next character.

Q7. When does left move?

Answer. When the new character violates uniqueness, move it until validity is restored, or jump using last-seen indices.

Q8. Why only forward?

Answer. Once data is excluded, moving backward would reintroduce it and destroy monotonic progress.

Q9. Time?

Answer. $O(n)$.

Q10. Space?

Answer. $O(k)$ for tracked distinct characters.

Q11. Common set bug?

Answer. Removing too little or too much while shrinking.

Q12. Index-map optimization?

Answer. Store last positions and jump left to one past the previous occurrence, while never moving left backward.

Q13. Important edge cases?

Answer. Empty string, one character, all unique, all identical, and repeated characters separated by other characters.

Q14. Correctness explanation?

Answer. Every active window is valid by the invariant; every right boundary is considered; contraction restores validity; the maximum valid length is therefore recorded.

Q15. Google-style answer?

Answer. Recognize contiguous substring + uniqueness, maintain a valid window, expand right, contract/jump left on duplicates, and use monotonic pointers for $O(n)$.

Part XVII — Maximum Average Subarray I: Follow-ups

Q1. What is the invariant?

Answer. `current_sum` always represents exactly the `k` elements in the current window.

Q2. What if all numbers are negative?

Answer. Initialize the best from the first valid window; never assume zero is a valid lower bound.

Q3. What if $k=n$?

Answer. There is only one candidate window.

Q4. What if $k=1$?

Answer. Each element is its own window; choose the maximum.

Q5. Main implementation risk?

Answer. Off-by-one errors in outgoing/incoming positions.

Q6. Why divide only at the end?

Answer. Comparing sums is sufficient and avoids unnecessary repeated floating-point work.

Q7. What is the key lesson?

Answer. Derive a constant-time transition between adjacent fixed-size windows.

Part XVIII — Minimum Size Subarray Sum

Q1. What is the problem?

Answer. Find the minimum length of a contiguous subarray whose sum is at least a target.

Q2. Why variable Sliding Window?

Answer. Expand until valid, then contract to find the shortest valid window.

Q3. What assumption is required?

Answer. The standard linear solution relies on positive, or appropriately non-negative, values so the sum behaves monotonically under expansion/contraction.

Q4. What if sum is below target?

Answer. Expand the right boundary.

Q5. What if sum reaches target?

Answer. Record the length, then shrink from the left while the condition remains satisfied.

Q6. Complexity?

Answer. $O(n)$ time and $O(1)$ auxiliary space under the standard assumptions.

Q7. No-solution case?

Answer. Return the problem-specified sentinel, commonly 0.

Q8. Common bug?

Answer. Shrinking only once instead of repeatedly while the window remains valid.

Q9. What if negative values are allowed?

Answer. The standard Sliding Window proof breaks; a different technique such as prefix sums with an appropriate data structure may be needed.

Q10. Interview explanation?

Answer. Grow until valid, shrink aggressively while valid, and rely on positive values for monotonic pointer behavior.

Part XIX — Complexity

Q1. Complexity of Longest Substring?

Answer. $O(n)$ time and $O(k)$ auxiliary space.

Q2. Complexity of Maximum Average Subarray I?

Answer. $O(n)$ time and $O(1)$ auxiliary space.

Q3. Complexity of Minimum Size Subarray Sum?

Answer. $O(n)$ time and $O(1)$ auxiliary space under positive-value assumptions.

Q4. Why can two $O(n)$ algorithms differ in practice?

Answer. Constant factors, allocations, memory access, data structures, and runtime behavior differ.

Q5. What is $O(1)$ auxiliary space?

Answer. Constant extra memory independent of input size.

Q6. What is $O(k)$ space?

Answer. Extra memory grows with the number of distinct tracked elements.

Q7. Why is $O(n^2)$ costly?

Answer. Repeated pairings or rescans grow quadratically as input grows.

Q8. How justify linear pointers?

Answer. Each pointer moves forward at most n positions.

Q9. Why discuss complexity after correctness?

Answer. Correctness comes first; complexity describes the cost of a correct algorithm.

Q10. Interview habit?

Answer. State time, auxiliary space, and the reasoning behind both.

Part XX — Edge Cases

Q1. API collection edge cases?

Answer. Empty collection, one document, many documents, and behavior after deletion.

Q2. Single-document endpoint edge cases?

Answer. Existing ID, missing ID, malformed identifier if applicable, and schema boundaries.

Q3. POST edge cases?

Answer. Missing fields, wrong types, empty/disallowed values, extra fields according to policy, and minimal valid input.

Q4. DELETE edge cases?

Answer. Existing, missing, repeated deletion, and invalid ID.

Q5. Maximum Average edge case?

Answer. All negative numbers.

Q6. Longest Substring edge cases?

Answer. Empty input, immediate duplicates, all unique, all identical.

Q7. Minimum Size edge cases?

Answer. No solution, exact target, single element sufficient, entire array required.

Q8. Why do edge cases matter to pattern recognition?

Answer. They expose hidden assumptions, initialization errors, and invariant violations.

Q9. How systematically find them?

Answer. Check empty, one item, extremes, duplicates, all-equal, already-valid, never-valid, and boundary positions.

Q10. Interview standard?

Answer. Proactively name important edge cases instead of waiting for the interviewer.

Part XXI — Code Explanation

Q1. How explain Sliding Window before coding?

Answer. State the window, invariant, maintained state, expansion, contraction, answer update, and complexity.

Q2. How explain a repository method?

Answer. State its contract, inputs, outputs, missing-data behavior, and persistence effect.

Q3. How explain a FastAPI route?

Answer. State method/path, request model, service call, response model, and error/status behavior.

Q4. How explain a 404?

Answer. Route → service → repository → absent resource → application not-found condition → API 404.

Q5. How explain a test?

Answer. Describe the behavior protected and why the assertion matters.

Q6. How explain $O(n)$ pointers?

Answer. Each pointer moves monotonically and never goes backward.

Q7. How explain a bug?

Answer. Identify the violated contract/invariant, give a minimal failing input, explain the failure, and propose the smallest robust correction.

Q8. How discuss trade-offs?

Answer. State requirement, choice, benefit, cost, and what changes at larger scale.

Q9. What if you do not know?

Answer. State what you know, identify uncertainty, and reason from first principles; never fabricate.

Q10. Goal of interview explanation?

Answer. Make your reasoning observable: pattern, design, correctness, complexity, failure modes, and scale.

Part XXII — Engineering Decisions

Q1. Why not PostgreSQL immediately?

Answer. Day 4 is about boundaries, API behavior, validation, and tests. PostgreSQL is the next dependency after the repository seam is clear.

Q2. Why not Redis?

Answer. No Day 4 requirement needs caching, queues, rate limiting, or shared ephemeral state.

Q3. Why not Docker?

Answer. Containerization is valuable later; adding it now would distract from the current application-architecture objective.

Q4. Why not RAG or embeddings?

Answer. Reliable document ingestion and persistence are prerequisites. Retrieval built too early risks rework.

Q5. Why not LangChain immediately?

Answer. The roadmap prioritizes understanding the underlying architecture before adding abstraction frameworks.

Q6. What is technology collecting?

Answer. Adding tools for resume keywords rather than because a concrete requirement justifies them.

Q7. What is a good decision record?

Answer. Problem, constraints, options, chosen approach, trade-offs, and consequences.

Q8. When introduce a database?

Answer. When durability, shared state, queryability, concurrency, or scale requires it; in this roadmap, PostgreSQL is the next step.

Q9. How do tests influence architecture?

Answer. Testability favors explicit boundaries and reduces coupling.

Q10. Core Day 4 principle?

Answer. Build depth before breadth: make a small system clean, testable, and explainable before expanding the stack.

Part XXIII — Project-Based Google Follow-ups

Q1. How does the document service evolve into ingestion?

Answer. The CRUD foundation can later accept files, extract and clean text, chunk content, generate embeddings, and persist metadata/content without turning routes into an ingestion pipeline.

Q2. Where does RAG fit?

Answer. Above reliable document and retrieval foundations, progressing from dense retrieval to hybrid BM25+vector, reranking, metadata filtering, and query decomposition.

Q3. Why Recall@K, MRR, nDCG?

Answer. They evaluate retrieval quality from complementary perspectives: recall of relevant results, rank of the first relevant result, and graded ranked relevance.

Q4. Why is the repository boundary important for RAG?

Answer. Retrieval needs reliable access to documents, chunks, metadata, and embeddings; persistence details should remain behind a storage boundary.

Q5. How will requirements and source code enter?

Answer. They become knowledge sources that are extracted, normalized, chunked, embedded, and stored for later retrieval.

Q6. How support Requirement→Code→Test→Result traceability?

Answer. Maintain explicit identifiers and metadata linking requirements, source artifacts, tests, and execution results.

Q7. Why citation correctness?

Answer. An LLM can produce plausible text with irrelevant or nonexistent evidence; citation evaluation checks whether evidence supports the claim.

Q8. What is human-in-the-loop?

Answer. Human approval or review for risky, destructive, ambiguous, or low-confidence actions.

Q9. Future agent architecture?

Answer. Planner → Policy → Tool → Result → LLM, with validation, allowlists, timeouts, and iteration limits.

Q10. What must never be fabricated?

Answer. Metrics, evaluations, production performance, deployment status, or feature completion.

Part XXIV — System Design Follow-ups

Q1. How scale GET /documents?

Answer. Use pagination, indexes, connection pooling, and justified caching; at larger scale consider read replicas and partitioning where needed.

Q2. First bottleneck?

Answer. It depends on workload: database reads, ingestion CPU/IO, or retrieval infrastructure may dominate. Measure before optimizing.

Q3. Why not cache automatically?

Answer. Caching introduces invalidation, consistency, memory, and operational complexity.

Q4. What consistency issue can caching create?

Answer. Stale reads after updates or deletes; define invalidation or TTL behavior.

Q5. What is horizontal scaling?

Answer. Adding application instances to handle traffic. It requires shared state outside local process memory.

Q6. What happens to in-memory storage under scaling?

Answer. Each instance has separate state, so data can differ across replicas.

Q7. How handle database outage?

Answer. Use timeouts, bounded retries where justified, health/readiness signals, safe errors, logging/tracing, and recovery mechanisms.

Q8. What is a timeout?

Answer. A bound on how long an operation waits for a dependency before failing.

Q9. What is reliability?

Answer. The ability to perform intended functions consistently under expected conditions and failures.

Q10. How does observability apply?

Answer. Later track requests, latency, errors, traces, model/prompt/retrieval versions, tokens, tool calls, and costs. Day 4 should not claim those are complete.

Part XXV — Git and Engineering Practice

Q1. What Git commits fit Day 4?

Answer. Meaningful commits can separate listing, retrieval, deletion, tests, and Sliding Window work.

Q2. Why small commits?

Answer. They are easier to review, debug, revert, and understand.

Q3. What should README contain?

Answer. Problem, motivation, architecture, decisions, approach, actual results/experiments, deployment status, monitoring, and failure modes—without false completion claims.

Q4. Why treat repository state as evidence?

Answer. Career preparation must be grounded in real implementation. A planned feature is not a completed feature.

Q5. What should be documented?

Answer. API behavior, architecture boundaries, decisions, tests, DSA patterns, and the next dependency.

Q6. What is a bad commit?

Answer. A huge mixed change containing unrelated features, refactors, experiments, and generated files.

Q7. Why GitHub?

Answer. It provides durable evidence, collaboration, reviewability, and a portfolio history.

Q8. What must never be claimed?

Answer. Tests, metrics, deployments, cloud resources, or features that were not actually implemented and verified.

Q9. How does Git history help interviews?

Answer. It demonstrates incremental engineering and helps reconstruct design evolution.

Q10. Engineering habit?

Answer. Implement → test → commit → document.

Part XXVI — Scenarios

Q1. Invalid document data?

Answer. Pydantic rejects it at the boundary; normal service processing should not execute.

Q2. Missing document ID?

Answer. Repository reports absence, application communicates not-found, API returns 404.

Q3. Deleted then requested?

Answer. The subsequent GET should return the missing-resource behavior.

Q4. Repository replaced with PostgreSQL?

Answer. Change persistence implementation/configuration while preserving the repository/service contract.

Q5. SQL added to a route?

Answer. It couples transport to persistence and violates the intended architecture.

Q6. Raw exception returned to client?

Answer. It risks leaking details and creates an unstable error contract.

Q7. Two instances behave differently?

Answer. Local in-memory state is not shared; durable shared persistence is required.

Q8. Sliding Window is $O(n^2)$?

Answer. Inspect for repeated scans or expensive inner operations; maintain window state incrementally.

Q9. Minimum Size fails with negatives?

Answer. The standard monotonic-sum assumption is violated.

Q10. Many libraries, little code?

Answer. Technology collecting; the roadmap prioritizes working, tested systems.

Part XXVII — Mock Interview

Q1. Design the Day 4 backend in two minutes.

Answer. Present FastAPI as the HTTP boundary, Pydantic as the contract layer, DocumentService as the application layer, and DocumentRepository as the persistence boundary with in-memory storage. Mention CRUD-like document operations, tests, 404 behavior, and PostgreSQL as the next planned step.

Q2. Why is an empty collection 200?

Answer. The collection exists even when empty; absence of members is not absence of the collection.

Q3. Explain Sliding Window pattern choice.

Answer. These problems use contiguous ranges, so maintain a window and update its state incrementally. Fixed windows have constant length; variable windows use a validity invariant.

Q4. Longest Substring conceptually?

Answer. Maintain a unique-character window, expand right, contract or jump left on duplicates, track maximum length, $O(n)$ time.

Q5. Maximum Average conceptually?

Answer. Maintain a k -sized sum, subtract outgoing and add incoming, track maximum, divide by k , $O(n)/O(1)$.

Q6. Minimum Size conceptually?

Answer. Expand until target reached, then shrink while valid to minimize length; with positive values this is $O(n)/O(1)$.

Q7. What improve next?

Answer. PostgreSQL persistence, followed by ingestion/retrieval. Do not jump to RAG or agents before foundations are reliable.

Q8. Biggest Day 4 trade-off?

Answer. In-memory simplicity versus durability/shared state. The repository boundary captures the trade-off.

Q9. What monitor in production?

Answer. Request latency, errors, throughput, dependency health, traces, and later AI-specific model/retrieval/tool/cost metrics.

Q10. Final Day 4 message?

Answer. The goal is a clean, testable backend foundation plus strong Sliding Window reasoning, not a pile of technologies.

Appendix A — Rapid-Fire Revision

Give a 30–60 second answer to each prompt. Then expand with an example or trade-off.

1. Define separation of concerns.
2. Define dependency injection.
3. Define API contract.
4. Why use Pydantic?
5. What is a response model?
6. What is a service layer?
7. What is a repository?
8. Why avoid SQL in routes?
9. Why is in-memory storage temporary?
10. When is 404 correct?
11. Why is an empty collection not 404?
12. What is exception translation?
13. Unit test vs API test?
14. What should an API test protect?
15. What is Sliding Window?
16. Fixed vs variable window?
17. What is an invariant?
18. When should you suspect Sliding Window?
19. Why is Longest Substring $O(n)$?
20. Why is Maximum Average fixed-window?
21. Why does Minimum Size require positive values for the standard solution?
22. Two Pointers vs Sliding Window?
23. What is $O(1)$ auxiliary space?
24. Why initialize max from the first window?
25. What is idempotency?
26. Why does in-memory state fail under horizontal scaling?
27. Why add PostgreSQL next?
28. Why avoid premature Redis?
29. What should never be fabricated?
30. How do you explain a trade-off?
31. How do you debug a 404?
32. How do you debug DELETE followed by GET?
33. What makes an API production-ready?
34. What is pagination?

- 35. Authentication vs authorization?**
- 36. What is observability?**
- 37. What is human-in-the-loop?**
- 38. What is citation correctness?**
- 39. What is retrieval Recall@K?**
- 40. How do Git commits support engineering evidence?**

Appendix B — Top 20 Questions to Master

1. Explain API → Service → Repository and why the boundaries exist.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

2. Walk through POST /documents end to end.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

3. Walk through GET /documents/{id} for an existing document.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

4. Walk through GET /documents/{id} for a missing document.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

5. Why does GET /documents return an empty list rather than 404?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

6. Why separate DocumentCreate and DocumentResponse?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

7. Why should business logic not live in FastAPI routes?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

8. Why use an in-memory repository on Day 4?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

9. How would you replace it with PostgreSQL?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

10. What belongs in DocumentService?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

11. What belongs in DocumentRepository?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

12. How do you test the API contract?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

13. What is Sliding Window and when do you recognize it?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

14. Explain fixed vs variable windows.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

15. Explain the invariant for Longest Substring Without Repeating Characters.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

16. Explain the $O(n)$ fixed-window solution for Maximum Average Subarray I.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

17. Explain the positive-number assumption for Minimum Size Subarray Sum.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

18. Compare Two Pointers and Sliding Window.

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

19. How would you scale this document API?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

20. What evidence would you use to claim Day 4 is complete?

Mastery: answer precisely, connect to Day 4, state the trade-off/failure mode where relevant, and handle a follow-up.

Final Day 4 Mastery Standard

Backend Architecture. Draw and explain HTTP Client → FastAPI/API Layer → DocumentService → DocumentRepository → In-Memory Storage.

API Contracts. Explain POST /documents, GET /documents, GET /documents/{id}, DELETE /documents/{id}, success semantics, and 404 behavior.

Pydantic. Explain request/response schemas and why API models are separate from persistence models.

Service Layer. Identify business logic, thin controllers, and service-level testing.

Repository. Explain persistence abstraction and the path from in-memory storage to PostgreSQL.

Errors. Explain validation errors, 404, exception translation, safe public errors, and failure tests.

API Testing. Cover happy paths, validation, empty collections, missing resources, and deletion state transitions.

Sliding Window. Recognize contiguous-range problems, fixed/variable windows, invariants, and pointer movement.

Three DSA Problems. Implement and explain Longest Substring Without Repeating Characters, Maximum Average Subarray I, and Minimum Size Subarray Sum.

Two Pointers Comparison. Distinguish the broader Two Pointers family from Sliding Window.

Engineering Judgment. Justify why PostgreSQL is next and why premature infrastructure/AI frameworks are deferred.

Git & Evidence. Use meaningful commits, tests, documentation, and actual repository evidence.

Future Architecture. Connect Day 4 to ingestion, retrieval, RAG, agents, evaluation, observability, and production without claiming future phases are complete.

Day 4 → Day 5 dependency: PostgreSQL and durable persistence. Do not restart earlier material unless a concrete test, implementation defect, or interview-review gap requires it.